Knowledge Graph für Claude Code — Architektur, Tools & Dokumentationsstrategie

Dieses Dokument ist das Ergebnis eines Diskussion zwischen @Jan Tegtmeier und Claude über das beste Vorgehen, Code zu dokumentieren und für die Arbeit mit Claude Code (oder anderen) aufzubereiten.

TLDR

- Es gibt bereits Tools, die genau das machen. Diese werden in diesem Dokument vorgestellt
- Die Tools nutzen Wissen aus verschiedenen Quellen:
  - Per automatisch aus dem Quellcode extrahierten strukturelle Informationen
  - Von Menschen erstellter Dokumentation im Repo zu
    - Architecture Decision Records
    - Domain Glossar
    - Architektur-Constraints & Invarianten
    - Deployment & Infrastruktur-Kontext
    - Inline-Annotationen im Code sowie
    - Modul-Level CONTEXT.md Files.
- Es werden konkrete Empfehlungen und Beispiele für diese Dokumentation gegeben.

1. Das Gesamtbild: Zwei Welten verbinden

Ein Knowledge Graph für KI-gestütztes Coding besteht aus zwei fundamental verschiedenen Informationsschichten:

Schicht	Quelle	Extrahierbar?	Werkzeuge
Strukturelle Schicht	Quellcode	✅ Automatisiert	AST-Parser, LSP, statische Analyse
Semantische Schicht	Menschliches Wissen	❌ Muss dokumentiert werden	ADRs, Annotationen, CLAUDE.md

Das Ziel ist es, beide Schichten in einem einheitlichen, maschinenlesbaren Graphen zu vereinen.

---

2. Knowledge-Graph-Schema

2.1 Node-Typen

```yaml
nodes:
  # === AUTOMATISCH EXTRAHIERBAR ===
  - type: Module
    properties: [name, path, language, loc, complexity]

  - type: Class
    properties: [name, module, visibility, is_abstract, decorators]

  - type: Function
    properties: [name, class, signature, return_type, complexity, is_async]

  - type: Interface
    properties: [name, module, methods]

  - type: Type
    properties: [name, definition, generic_params]

  - type: Package
    properties: [name, version, source]  # externe Abhängigkeit

  - type: Config
    properties: [name, path, format]

  - type: Endpoint
    properties: [method, path, handler, auth_required]

  # === MANUELL DOKUMENTIERT ===
  - type: DomainConcept
    properties: [name, description, bounded_context]

  - type: ArchitectureDecision
    properties: [id, title, status, date, confidence]

  - type: Invariant
    properties: [description, severity, affected_components]

  - type: DeploymentUnit
    properties: [name, runtime, scaling_strategy]

  - type: ExternalSystem
    properties: [name, protocol, sla, owner]

  - type: SecurityBoundary
    properties: [name, trust_level, auth_mechanism]

```

2.2 Edge-Typen (Beziehungen)

```yaml
edges:
  # === AUTOMATISCH EXTRAHIERBAR ===
  - type: IMPORTS
    confidence: EXTRACTED

  - type: INHERITS_FROM
    confidence: EXTRACTED

  - type: IMPLEMENTS
    confidence: EXTRACTED

  - type: CALLS
```

```yaml
    properties: [is_async, frequency_hint]
    confidence: EXTRACTED

  - type: DEPENDS_ON
    properties: [version_constraint]
    confidence: EXTRACTED

  - type: CONTAINS      # Module → Class → Function
    confidence: EXTRACTED

  - type: RETURNS_TYPE
    confidence: EXTRACTED

  - type: THROWS
    confidence: EXTRACTED

  # === TEILWEISE EXTRAHIERBAR (aus Docstrings, Kommentaren) ===
  - type: REPRESENTS     # Class → DomainConcept
    confidence: INFERRED

  - type: TRANSFORMS     # Function transformiert Daten von A → B
    confidence: INFERRED

  # === NUR MANUELL ===
  - type: DECIDED_BY     # Component → ArchitectureDecision
    confidence: MANUAL

  - type: MUST_SATISFY   # Component → Invariant
    confidence: MANUAL

  - type: DEPLOYS_TO     # Module → DeploymentUnit
    confidence: MANUAL

  - type: COMMUNICATES_WITH  # Service → ExternalSystem
    properties: [protocol, is_sync, retry_policy]
    confidence: MANUAL

  - type: PROTECTED_BY   # Endpoint → SecurityBoundary
    confidence: MANUAL

  - type: REPLACES       # ArchitectureDecision → ArchitectureDecision
    confidence: MANUAL

```

2.3 Provenance-Markierungen

Jede Kante erhält eine Confidence-Markierung:

- **EXTRACTED** — deterministisch aus AST/Code extrahiert, 100% korrekt
- **INFERRED** — durch LLM oder Heuristik abgeleitet, Review empfohlen
- **MANUAL** — von Menschen dokumentiert
- **AMBIGUOUS** — unsicher, Verifikation nötig

---

3. Automatisierte Extraktion: Existierende Tools

3.1 Graphify (empfohlen für Claude Code)

Das vielversprechendste Tool für diesen Use Case. Graphify ist ein Open-Source-Skill, der speziell für KI-Coding-Assistenten gebaut wurde.

Was es tut:

- Zwei-Pass-Pipeline: erst deterministischer AST-Pass über Code, dann LLM-Subagenten für Docs, Papers, Screenshots und andere Nicht-Code-Artefakte
- Tree-sitter-Parsing für 20 Sprachen (Python, JS, TS, Go, Rust, Java, C, C++ u.v.m.)
- Multimodal: verarbeitet Code, Markdown, PDFs, Screenshots, Whiteboard-Fotos
- Provenance-Tagging: jede Beziehung wird als EXTRACTED, INFERRED oder AMBIGUOUS markiert
- Gibt JSON-Graph, Markdown-Report und HTML-Viewer aus
- Laut Projekt 71,5x weniger Tokens pro Query als Rohdateien lesen

Installation als Claude Code Skill:

```
# Im Projektverzeichnis
graphify build
# Erzeugt: graph.json, GRAPH_REPORT.md, graph_viewer.html

```

Quelle: Graphify — Knowledge Graph Skill for AI Coding Assistants

3.2 CodePrism + Mandrel MCP

Ein Rust-basiertes Tool, das einen Universal AST über die gesamte Codebasis baut.

Was es tut:

- Baut einen sprachübergreifenden Graphen (nicht dateiweise, sondern systemweit)
- Versteht Cross-Language-Beziehungen
- Indexiert 1000+ Dateien pro Sekunde
- Verfügbar als MCP-Server für Claude Code

Besonderheit: Verfolgt den Ansatz "Denken in Graphen statt in Bäumen" — statt einzelne ASTs pro Datei zu analysieren, wird ein vereinigter Graph gebaut, der Beziehungen über Dateigrenzen hinweg versteht.

Quelle:

https://github.com/nicobailon/codeprism (nicht mehr verfügbar!)

3.3 GitNexus

Client-seitig, läuft komplett im Browser.

Was es tut:

- Sieben-Phasen-Indexierung: Structure → Parse → Imports → Calls → Heritage → Communities → Processes
- Impact-Analyse: zeigt, welche Komponenten betroffen sind, wenn du etwas änderst
- Community Detection: erkennt Cluster zusammengehöriger Code-Teile
- Verfügbar als MCP-Server (gitnexus-mcp)

Besonderheit: Die Community-Erkennung ist besonders nützlich, weil sie funktionale Cluster identifiziert, die nicht unbedingt der Verzeichnisstruktur entsprechen.

Quelle: GitHub - barrulus/gitnexus: GitNexus: The Zero-Server Code Intelligence Engine - GitNexus is a client-side knowledge graph creator that runs entirely in your browser. Drop in a GitHub repo or ZIP file, and get an interactive knowledge graph wit a built in Graph RAG Agent. Perfect for code exploration

3.4 Code Grapher (Neo4j-basiert)

Für große, produktive Codebasen mit Datenbank-Backend.

Was es tut:

- AST-Parsing mit Neo4j als Graph-Datenbank
- Hybride Suche: Vektor-Ähnlichkeit (ChromaDB) + Graph-Traversal (Neo4j)
- Natural Language Queries: "Wie funktioniert die Authentifizierung?"
- PRIMER.md-Dateien für Business-Kontext-Injection
- MCP-Server für Claude-Integration

Besonderheit: Unterstützt `PRIMER.md`-Dateien, die Domain-Wissen automatisch in die KI-Beschreibungen einfließen lassen — das ist konzeptionell genau dein Ansatz.

Quelle:

https://github.com/nicobailon/code-grapher (nicht mehr verfügbar!)

3.5 Klassische statische Analyse (als Basis)

Für gezielte Aspekte können auch Einzeltools kombiniert werden:

Tool	Sprache	Extrahiert

tree-sitter	20+ Sprachen	AST, Symbole, Strukturen
ast-grep	Multi-Language	Strukturelle Code-Patterns
madge	JavaScript/TS	Modul-Abhängigkeitsgraphen
pydeps	Python	Package-Abhängigkeiten
jdepend / ArchUnit	Java	Package-Zyklen, Architektur-Constraints
Sourcetrail (archiviert)	C/C++/Java/Python	Interaktiver Quellcode-Explorer
Doxygen	Multi-Language	Dokumentation + Call-Graphen

---

4. Was NICHT automatisch extrahierbar ist — und wo es dokumentiert werden muss

Hier liegt der eigentliche Mehrwert deiner Arbeit. Automatisierte Tools liefern das "Was" und "Wie" — du musst das "Warum", "Wozu" und "Warum nicht anders" dokumentieren.

4.1 Architecture Decision Records (ADRs)

Speicherort im Repo: `docs/adr/` oder `docs/decisions/`

ADRs dokumentieren architekturrelevante Entscheidungen mit ihrem Kontext und ihren Konsequenzen.

Empfohlenes Format (MADR):

```
<!-- docs/adr/003-kafka-statt-rabbitmq.md -->

# ADR-003: Kafka statt RabbitMQ für Event-Streaming

## Status
Accepted (2025-03-15)

## Kontext
Wir brauchen eine Event-Streaming-Lösung für die Kommunikation
zwischen OrderService und InventoryService. Die Events müssen
replay-fähig sein, da der InventoryService geplante Downtimes hat.

## Entscheidung
Wir verwenden Apache Kafka.

## Begründung
- RabbitMQ verliert Messages nach ACK – kein Replay möglich
- Kafka's Log-basiertes Modell erlaubt Consumer-Restart ab Offset
- Team hat bereits Kafka-Erfahrung aus Projekt X

```

```
## Verworfene Alternativen
- **RabbitMQ**: Kein nativer Replay-Support
- **AWS SQS**: Vendor Lock-in, max 14 Tage Retention

## Konsequenzen
- Zusätzliche Infrastruktur-Komplexität (ZooKeeper/KRaft)
- Schema-Registry wird benötigt (→ ADR-004)
- InventoryService muss idempotent verarbeiten

## Betroffene Komponenten
- OrderService (Producer)
- InventoryService (Consumer)
- NotificationService (Consumer)
- Infrastructure/docker-compose.yml

```

Für den Knowledge Graph: Jede ADR wird ein `ArchitectureDecision`-Node mit `DECIDED_BY`-Kanten zu den betroffenen Komponenten.

4.2 Domain-Glossar

Speicherort im Repo: `docs/domain/glossary.yaml`

```yaml
# docs/domain/glossary.yaml
domain_concepts:
  - name: Order
    description: "Eine Kundenbestellung mit einem oder mehreren LineItems"
    bounded_context: Sales
    represented_by:
      - src/services/order/Order.ts
      - src/services/order/OrderAggregate.ts
    invariants:
      - "Order darf nicht bestätigt werden ohne mindestens ein LineItem"
      - "Gesamtbetrag muss immer aus LineItems berechnet werden, nie direkt gesetzt"

  - name: Fulfillment
    description: "Der Prozess von Bestellbestätigung bis Versand"
    bounded_context: Logistics
    represented_by:
      - src/services/fulfillment/FulfillmentProcess.ts
    depends_on_concepts:
      - Order
      - InventoryReservation
    lifecycle:
      - PENDING → PICKING → PACKING → SHIPPED → DELIVERED

```

4.3 Architektur-Constraints & Invarianten

Speicherort im Repo: `docs/architecture/constraints.yaml`

```yaml
# docs/architecture/constraints.yaml
constraints:
  - id: C-001
    description: "Services dürfen nicht direkt auf die Datenbank anderer Services zugreifen"
    type: boundary
    severity: critical
    rationale: "Microservice-Autonomie — siehe ADR-001"
```

```yaml
    enforced_by: "ArchUnit-Tests in test/architecture/"

  - id: C-002
    description: "Alle API-Endpoints müssen durch das AuthGateway geroutet werden"
    type: security
    severity: critical
    exceptions:
      - "/health"
      - "/metrics"

  - id: C-003
    description: "Maximale Antwortzeit für User-facing APIs: 200ms (P99)"
    type: performance
    severity: high
    measurement: "Grafana Dashboard 'API Latency'"

```

4.4 Deployment & Infrastruktur-Kontext

Speicherort im Repo: `docs/architecture/deployment.yaml`

```yaml
# docs/architecture/deployment.yaml
deployment_units:
  - name: order-service
    runtime: Node.js 20 (Docker)
    scaling: horizontal, 2-10 Instanzen
    database: PostgreSQL 15 (AWS RDS)
    deploys_modules:
      - src/services/order/
    communicates_with:
      - target: kafka-cluster
        protocol: Kafka
        topics: [orders.created, orders.updated]
      - target: payment-gateway
        protocol: REST/HTTPS
        is_external: true
        sla: "99.9%, max 500ms"
        retry_policy: "3x exponential backoff"
    environment_vars:
      - DATABASE_URL (Secret)
      - KAFKA_BROKERS
      - PAYMENT_API_KEY (Secret)

```

4.5 Inline-Annotationen im Code

Speicherort: Direkt im Quellcode, maschinenlesbar

Für Informationen, die nah am Code bleiben müssen, sind strukturierte Kommentare der beste Ort:

```
/**
 * @domain Order
 * @bounded-context Sales
 * @invariant Gesamtbetrag muss immer aus LineItems berechnet werden
 * @decision ADR-007
 * @security Nur authentifizierte Benutzer mit Rolle SALES_AGENT
 * @perf Hot Path — keine N+1 Queries, immer eager-load LineItems
 */
export class OrderService {
```

```

  /**
   * @why Wir verwenden hier optimistic locking statt pessimistic,
   *   weil parallele Bestelländerungen selten sind (<0.1%)
   *   und pessimistic locking den Payment-Flow blockieren würde.
   * @see ADR-012
   */
  async updateOrder(id: string, changes: OrderChanges): Promise<Order> {
    // ...
  }
}

```

Diese Tags können von einem Extraktions-Script geparst werden und als `MANUAL`-Kanten in den Knowledge Graph fließen.

4.6 Modul-Level CONTEXT.md

Speicherort: In jedem wichtigen Unterverzeichnis

```
<!-- src/services/payment/CONTEXT.md -->

# Payment Service — Architekturkontext

## Verantwortlichkeit
Kapselt alle Interaktionen mit externen Payment Providern.
Kein anderer Service darf direkt mit Payment-APIs kommunizieren.

## Warum diese Struktur?
Der PaymentAdapter ist absichtlich als Strategy Pattern implementiert
(nicht als einfacher Service), weil wir zwischen Stripe (EU) und
Adyen (APAC) wechseln — gesteuert durch die Region des Kunden.
Siehe ADR-009.

## Bekannte technische Schulden
- [ ] Retry-Logik ist noch im Controller statt im Adapter (Tech-Debt #142)
- [ ] Webhook-Validation fehlt für Adyen (Security-Risk, Prio High)

## Nicht-offensichtliche Abhängigkeiten
- Der PaymentService wird vom OrderService UND vom SubscriptionService
  verwendet. Änderungen am Interface brechen beide.
- Die Webhook-Endpoints müssen public erreichbar sein (Ausnahme von C-002)

```

---

5. Die Extraktions-Pipeline

5.1 Empfohlener Workflow

```
┌──────────────────────────────────────────────────────────┐
│  Phase 1: Automatische Extraktion                        │
│                                                          │
│  Quellcode ──→ Tree-sitter/AST ──→ Struktureller Graph   │
│  (Classes, Functions, Imports, Calls, Types)             │
│                                                          │
│  Tool: graphify / CodePrism / tree-sitter direkt         │
│  Confidence: EXTRACTED                                   │
└──────────────────────────────────────────────────────────┘
                          │
                          │
```

```
                        ▼
┌──────────────────────────────────────────────────────────┐
│  Phase 2: Semantische Anreicherung                       │
│                                                          │
│  Docstrings, Kommentare, README ──→ LLM ──→ Konzepte    │
│  @domain / @why / @invariant Tags ──→ Parser ──→ Kanten  │
│                                                          │
│  Tool: graphify (LLM-Pass) / eigenes Script              │
│  Confidence: INFERRED                                    │
└──────────────────────────────────────────────────────────┘
                        │
                        ▼
┌──────────────────────────────────────────────────────────┐
│  Phase 3: Manuelle Wissensintegration                    │
│                                                          │
│  docs/adr/*.md          ──→ ArchitectureDecision-Nodes   │
│  docs/domain/glossary   ──→ DomainConcept-Nodes          │
│  docs/architecture/*    ──→ Constraint/Deployment-Nodes  │
│  */CONTEXT.md           ──→ Enrichment bestehender Nodes │
│                                                          │
│  Tool: YAML/Markdown-Parser (eigenes Script)             │
│  Confidence: MANUAL                                      │
└──────────────────────────────────────────────────────────┘
                        │
                        ▼
┌──────────────────────────────────────────────────────────┐
│  Phase 4: Graph-Ausgabe                                  │
│                                                          │
│  → graph.json          (maschinenlesbar)                 │
│  → GRAPH_REPORT.md     (für CLAUDE.md / Plan-Modus)      │
│  → graph_viewer.html   (für menschliche Inspektion)      │
└──────────────────────────────────────────────────────────┘
```

5.2 Graph-Verdichtung für den Plan-Modus

Das ist der Punkt, an dem dein Ansatz seinen vollen Wert entfaltet. Statt Claude den gesamten Graphen zu geben, extrahierst du einen **Feature-relevanten Subgraphen:**

```
Eingabe:
  - Vollständiger Knowledge Graph (graph.json)
  - Feature-Beschreibung: "Neues Feature: Kunden können Bestellungen stornieren"

Verdichtungs-Prompt an Claude:
  "Analysiere den Knowledge Graph und identifiziere alle Nodes und Edges,
   die für das Feature 'Bestellstornierung' relevant sind. Berücksichtige:
   1. Direkt betroffene Komponenten
   2. Transitive Abhängigkeiten (max. 2 Hops)
   3. Relevante Architektur-Entscheidungen und Constraints
   4. Domain-Invarianten, die verletzt werden könnten
   5. Deployment-Auswirkungen

   Gib den verdichteten Subgraphen als YAML aus."

Ergebnis:
  - Fokussierter Subgraph mit ~20-50 Nodes statt ~500+
  - Enthält das "Warum" (ADRs, Constraints)
  - Enthält versteckte Abhängigkeiten (Payment-Webhooks!)
  - Passt in den Kontext des Plan-Modus
```

6. Repo-Struktur: Wo alles hingehört

```
project-root/
├── CLAUDE.md                          # Verweis auf Knowledge Graph
├── docs/
│   ├── adr/                           # Architecture Decision Records
│   │   ├── 001-microservice-split.md
│   │   ├── 002-event-sourcing.md
│   │   └── ...
│   ├── domain/
│   │   ├── glossary.yaml              # Domain-Konzepte & Bounded Contexts
│   │   └── event-flows.yaml           # Fachliche Event-Ketten
│   ├── architecture/
│   │   ├── constraints.yaml           # Architektur-Regeln
│   │   ├── deployment.yaml            # Infrastruktur-Kontext
│   │   ├── security-boundaries.yaml   # Trust Levels & Auth
│   │   └── tech-debt.yaml             # Bekannte Schulden
│   └── knowledge-graph/
│       ├── graph.json                 # Vollständiger Graph (auto-generiert)
│       ├── GRAPH_REPORT.md            # Lesbare Zusammenfassung
│       └── build-graph.sh             # Extraktions-Pipeline
├── src/
│   ├── services/
│   │   ├── order/
│   │   │   ├── CONTEXT.md             # Modul-spezifischer Kontext
│   │   │   ├── Order.ts               # Mit @domain, @why Tags
│   │   │   └── ...
│   │   ├── payment/
│   │   │   ├── CONTEXT.md
│   │   │   └── ...
│   │   └── ...
│   └── ...
└── ...
```

---

7. Zusammenfassung: Was automatisiert geht, was nicht

Information	Automatisiert?	Tool	Dokumentiert in
Klassen & Funktionen	✅	Tree-sitter / AST	—
Import-Beziehungen	✅	Tree-sitter / AST	—
Vererbung & Interfaces	✅	Tree-sitter / AST	—
Call-Graph	✅	AST + statische Analyse	—
Typ-Informationen	✅	LSP / TypeScript Compiler	—
Package-Abhängigkeiten	✅	package.json / requirements.txt	—

API-Endpoints	✅	AST (Decorator-Erkennung)	—
Code-Komplexität	✅	Statische Analyse	—
Domain-Konzepte	⚠️ teilweise	LLM aus Docstrings	`docs/domain/glossary.yaml`
Architektur-Entscheidungen	❌	—	`docs/adr/*.md`
Invarianten & Constraints	❌	—	`docs/architecture/constraints.yaml`
Deployment-Kontext	❌	—	`docs/architecture/deployment.yaml`
Security-Boundaries	❌	—	`docs/architecture/security-boundaries.yaml`
Technische Schulden	❌	—	`docs/architecture/tech-debt.yaml`
Fachliche Event-Flows	❌	—	`docs/domain/event-flows.yaml`
"Warum so und nicht anders"	❌	—	`docs/adr/*.md` + `*/CONTEXT.md`
Externe System-SLAs	❌	—	`docs/architecture/deployment.yaml`
Nicht-offensichtliche Abhängigkeiten	❌	—	`*/CONTEXT.md` + Inline-Tags